

## Desafio técnico — API de Empréstimos Concorrentes e Auditáveis

### Objetivo

Implemente uma API REST em .NET 10 LTS para o catálogo e os empréstimos de uma biblioteca. O objetivo é avaliar decisões de engenharia em um serviço distribuído: integridade dos dados, concorrência, rastreabilidade, cache, testes e operação.

O escopo é intencionalmente limitado. Priorize corretude, clareza e testes em vez de quantidade de endpoints ou complexidade de arquitetura.

### Regras de entrega

Tempo esperado: 5 a 7 horas.

Entregue um repositório Git com código-fonte, histórico de commits e README.

O projeto deve iniciar localmente com Docker Compose, incluindo PostgreSQL e Redis.

Não é necessário possuir conta Azure, cluster Kubernetes ou conta DataDog.

Uso de ferramentas de IA é permitido, desde que você consiga explicar e defender todas as decisões e o código entregue.

### Stack obrigatória

.NET 10 LTS;

ASP.NET Core Web API;

PostgreSQL;

Entity Framework Core com provider Npgsql;

Redis;

testes automatizados;

Dockerfile;

manifests Kubernetes ou Helm chart básico.

O .NET 10 é uma versão LTS ativa. Consulte a política oficial de suporte .NET.

Você pode escolher a organização interna do projeto e bibliotecas auxiliares. A adoção de Clean Architecture, DDD, CQRS ou qualquer outro padrão não será avaliada isoladamente; as decisões precisam resolver problemas concretos do desafio.

### Domínio

Uma biblioteca possui livros e vários exemplares de cada livro. Usuários podem consultar a disponibilidade e realizar empréstimos.

A API será executada em múltiplas réplicas. Portanto, duas requisições

simultâneas não podem emprestar o mesmo último exemplar. Todo empréstimo e toda mudança relevante devem manter histórico auditável.

## Requisitos funcionais

### 1. Catálogo

Implemente operações para:

criar, consultar, atualizar e listar livros;

identificar cada livro por título, ISBN, autor e quantidade de exemplares;

garantir unicidade de ISBN;

desativar ou rejeitar a remoção de um livro com empréstimos/histórico. O histórico não pode ser apagado silenciosamente.

### 2. Usuários e empréstimos

Implemente operações para:

cadastrar um usuário/leitor;

realizar empréstimo de um exemplar disponível;

registrar data do empréstimo e data prevista de devolução;

devolver um empréstimo;

cancelar um empréstimo, preservando a informação de que ele existiu;

consultar disponibilidade atual de um livro;

consultar histórico de empréstimos por livro e por usuário.

### 3. Concorrência e consistência

Se houver apenas um exemplar disponível, duas requisições concorrentes para emprestá-lo devem produzir:

exatamente um empréstimo bem-sucedido;

uma rejeição clara de regra de negócio para a outra tentativa;

nenhuma quantidade negativa, empréstimo duplicado ou estado inconsistente.

Implemente e automatize esse cenário em um teste de integração. Documente no README a estratégia adotada — por exemplo, atualização condicional atômica, lock de linha, token de concorrência ou isolamento de transação — e os trade-offs considerados.

Não use ``rowversion`` como pressuposto: esse recurso é específico do SQL Server. Para PostgreSQL, a estratégia deve ser apropriada ao banco utilizado. Veja a documentação do EF Core sobre conflitos de concorrência e a documentação de isolamento de transações do PostgreSQL.

#### 4. Idempotência

O endpoint de criação de empréstimo deve aceitar o cabeçalho `Idempotency-Key`.

Repetir a mesma requisição com a mesma chave não pode criar dois empréstimos nem reduzir a disponibilidade duas vezes. A API deve devolver a resposta previamente produzida ou resultado semanticamente equivalente.

#### 5. Auditoria de domínio

Registre eventos de auditoria para, no mínimo:

criação e alteração de livro;

desativação de livro;

criação de empréstimo;

devolução de empréstimo;

cancelamento de empréstimo.

Cada evento deve registrar entidade, identificador, ação, ator, timestamp em UTC, identificador de correlação e informação suficiente para entender a mudança. O objetivo é preservar uma trilha de negócio; logs técnicos não a substituem.

#### 6. Cache

Use Redis em pelo menos uma consulta de leitura, como catálogo ou disponibilidade. Quando um empréstimo, devolução ou alteração de quantidade afetar esses dados, o cache deve ser invalidado ou atualizado de modo coerente. O PostgreSQL deve permanecer como fonte de verdade para decisão de disponibilidade e criação de empréstimos.

#### Requisitos de operação

##### API e configuração

Use operações assíncronas e propague `CancellationToken` quando aplicável.

Utilize status HTTP adequados e respostas de erro consistentes, preferencialmente Problem Details.

Configure a aplicação por `appsettings.\*.json` e variáveis de ambiente.

Não versione segredos, connection strings com credenciais ou chaves reais.

Versione as migrations do Entity Framework Core.

#### Health checks e telemetria

Implemente:

`GET /health/live` : confirma que o processo está ativo; não deve falhar apenas

porque PostgreSQL ou Redis estão indisponíveis;  
`GET /health/ready` : confirma que as dependências necessárias para servir tráfego estão disponíveis;  
logs estruturados para operações relevantes, incluindo `correlationId` e os identificadores de negócio pertinentes;  
métricas de empréstimos criados, rejeições por indisponibilidade, operações idempotentes repetidas e latência do endpoint de empréstimo;  
instrumentação de traces e métricas compatível com OpenTelemetry.  
O ASP.NET Core fornece middleware próprio para health checks, que pode ser segmentado por endpoint e usado por orquestradores de contêineres.  
Documentação Microsoft. OpenTelemetry é um padrão vendor-neutral para geração e exportação de traces, métricas e logs. Documentação OpenTelemetry.

## Kubernetes

Inclua manifests ou Helm chart com, no mínimo:

`Deployment` ;

`Service` ;

referências a configuração e segredos sem valores reais;

requests e limits de CPU/memória;

liveness e readiness probes.

No README, explique como a aplicação permanece correta entre 2 e 11 réplicas.

Não é necessário provisionar ou executar um cluster.

## Endpoints mínimos sugeridos

Você pode ajustar o contrato quando necessário, desde que documente a decisão.

```text

POST /books

GET /books

GET /books/{id}

PATCH /books/{id}

DELETE /books/{id}

POST /users

GET /users/{id}/loans

POST /loans # exige Idempotency-Key  
POST /loans/{id}/return  
POST /loans/{id}/cancel

GET /books/{id}/availability  
GET /books/{id}/history  
GET /audit-events  
GET /health/live  
GET /health/ready  
...

#### Testes mínimos

testes unitários das regras de negócio;  
testes de integração com PostgreSQL real ou containerizado;  
teste concorrente para o último exemplar disponível;  
teste de idempotência;  
teste que comprove a preservação de histórico após devolução/cancelamento.

#### README esperado

Inclua:

pré-requisitos e comandos de execução;  
como executar migrations e testes;  
variáveis de ambiente necessárias, sem segredos;  
decisões de concorrência, idempotência, cache e auditoria;  
limitações conhecidas e como você evoluiria a solução para produção.  
Boa implementação.